

P1028R5: SG14 `status_code` and standard `error` object

Document #: P1028R5
Date: 2023-05-11
Project: Programming Language C++
Library Evolution Working Group
Reply-to: Niall Douglas
<s_sourceforge@nedprod.com>

A proposed wording for the replacement, in new code, of the system header `<system_error>` with a substantially refactored and lighter weight design, which meets modern C++ design and implementation. In the Issaquah 2023 meeting, LEWG requested IS wording for this proposal targeting the C++ 26 standard release.

You can find the arguments about design rationale in R4 (<https://wg21.link/P1028R4>). From R5 onwards, this has been condensed into a set of design goals and change tracking log.

A C++ 11 reference implementation of the proposed replacement can be found at <https://github.com/ned14/status-code>. Support for the proposed objects has been wired into Boost.Outcome [1] which has been shipping with the Boost C++ Libraries for four years.

The reference implementation has been found to work well on recent editions of GCC, clang and Microsoft Visual Studio, on x86, x64, ARM and AArch64. It has been quite popular with the C++ userbase, indeed there are two known complete re-implementations, one of which was described by [P2170] *Feedback on implementing the proposed std::error type*. This proposed design has shipped on every recent copy of Microsoft Windows and Apple iOS, and a fair chunk of Android devices. I believe it is amongst the best tested designs proposed for library standardisation in recent years.

This proposal is a much richer and more powerful framework than `<system_error>`, whilst remaining fully backwards compatible with it. Indeed, it can almost completely replace the dynamic exception mechanism with a fully deterministic alternative, and it has been proposed as the `std::error` implementation for [P0709] *Zero overhead deterministic exceptions* in [P1095] *Zero overhead deterministic failure*.

Contents

1 Design goals	2
2 Change tracking log for LWG since R4	3
3 Delta from N4928	3
4 Acknowledgements	38
5 References	39

1 Design goals

These were originally set by SG14, and have to date survived LEWG discussion of this paper. You can read their original paper [P0824] *Summary of SG14 discussion on <system_error>* for more detail.

1. **Do not cause `#include <string>`.**

Summary of P0824 rationale: No need to be dragging in string handling, the allocator machinery et al. This proposal is designed to work well on Freestanding.

2. **All `constexpr` sourcing, construction and destruction.**

Summary of P0824 rationale: `std::error_category` is impossible to optimise out, and thus introduces unnecessary runtime overhead every time a `std::error_code` is constructed.

3. **Header only libraries can now safely define custom code categories.**

Summary of P0824 rationale: `std::error_category` is completely broken in header only libraries, and can never work reliably at all.

4. **No more `if(!ec)...`**

Summary of P0824 rationale: That boolean test is unreliable/confusing and doesn't do what most developers think it does.

5. **No more filtering codes returned by systems APIs.**

Summary of P0824 rationale: Stop special casing/working around all bits zero error code values.

6. **All comparisons between codes are now semantic, not literal.**

Summary of P0824 rationale: `std::error_code` comparison semantics are inconsistently confusing even for the expert.

7. **`std::error_condition` is removed entirely (i.e. not modelled in this proposal, though we retain 100% compatibility with it).**

Summary of P0824 rationale: `std::error_code` comparison semantics are inconsistently confusing even for the expert.

8. **`status_code`'s value type is set by its domain.**

Summary of P0824 rationale: `std::error_code`'s value type is hard coded to an `int` which is limiting. `status_code` can carry any value of any arbitrary type `T`.

9. **`status_code<DomainType>` is type erasable.**

Summary of P0824 rationale: If a `status_code`'s arbitrary value type `T` can be safely bit copied (e.g. is trivially copyable), we allow conversion to an `erased_status_code`.

10. **More than one 'system' error coding domain: `system_code`.**

Summary of P0824 rationale: Platforms have multiple system error coding schemas, and this ought to be modelled into standard C++ instead of claiming that there is only one ‘system’ coding domain (`std::system_category`).

11. `std::errc` gets its own code domain `generic_code`, eliminating modelling `std::error_condition`.

Summary of P0824 rationale: By giving `std::errc` its own domain, we regularise semantic comparisons between unknown status code types into a consistent logic with predictable meanings and outcomes.

2 Change tracking log for LWG since R4

The WG21 tracker for this paper can be found at <https://github.com/cplusplus/papers/issues/405>.

- R4 => R5:
 - `string_ref` => ‘implementation defined type’ as per LEWG request.
 - `status_code_ptr` => `nested_status_code` as per LEWG request.
 - `make_nested_status_code()` now takes a STL allocator, as per LEWG request.
 - `erased<T>` has been removed as per LEWG request. A new type alias `erased_status_code<T>` alias to the appropriate tagged specialisation of `status_code`.

3 Delta from N4928

The following normative wording delta is against <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4928.pdf>. For existing normative wording, green text is wording to be added, red text is wording to be removed, black text is generally notes to LEWG which shall be removed if the paper is sent to LWG. For the entirely new sections, assume it as if it were all green.

In 17.3.2 [version.syn] paragraph 2:

```
#define __cpp_lib_system_status 202305L //also in <system_status>
```

In 19.5.2 [system.error.syn]:

```
enum class errc {  
    unknown = -1,  
    success = 0,  
    address_family_not_supported, //EAFNOSUPPORT
```

[Note: It is suggested that the following sections be appended after 19.6 [stacktrace] as section 19.7 – end note]

System status support [sysstatus]

General [sysstatus.general]

Subclause 19.7 describes components that the standard library and C++ programs may use to report error, warning, informational and success conditions originating from code behind an opaque ABI boundary, which includes the operating system. It is a fully compatible superset of the components within 19.5 [syserr], and is intended to completely replace those in newly written code.

Header <system_status> [system.status.syn]

Class `status_code_domain` [system.status.code__domain]

An implementation of abstract base class `status_code_domain` defines the coding and interpretation of status codes of that domain.

`status_code_domain` is trivially copyable, and can be constructed and destructed at compile-time.

```
1 namespace std {
2     template <class DomainType> class status_code;
3     class _generic_code_domain;
4     using generic_code = status_code<_generic_code_domain>;
5
6     class status_code_domain {
7     template <class DomainType> friend class status_code;
8     template <class StatusCode> friend class indirecting_domain;
9 public:
10     using unique_id_type = /* implementation defined unsigned, at least 64 bits */;
11
12     class string_ref; // implementation defined
13     class atomic_refcounted_string_ref; // implementation defined, refines string_ref
14
15     struct payload_info_t {
16         size_t payload_size{0}; // The payload size in bytes
17         size_t total_size{0}; // The total status code size in bytes (includes domain pointer and
18                               mixins state)
19         size_t total_alignment{0}; // The total status code alignment in bytes
20
21         payload_info_t() = default;
22         constexpr payload_info_t(size_t _payload_size, size_t _total_size, size_t _total_alignment);
23     };
24 private:
25     const unique_id_type _id; // exposition only
26
27 protected:
28     constexpr explicit status_code_domain(unique_id_type id) noexcept;
29     template<size_t N>
30     constexpr explicit status_code_domain(const char (&uuid)[N]) noexcept;
```

```

30     template<size_t N>
31     constexpr explicit status_code_domain(const char* uuid, _uuid_size<N>) noexcept;
32
33     // All trivial
34     status_code_domain(const status_code_domain &) = default;
35     status_code_domain(status_code_domain &&) = default;
36     status_code_domain &operator=(const status_code_domain &) = default;
37     status_code_domain &operator=(status_code_domain &&) = default;
38     ~status_code_domain() = default;
39
40 public:
41     constexpr bool operator==(const status_code_domain &o) const noexcept;
42     constexpr bool operator!=(const status_code_domain &o) const noexcept;
43     constexpr bool operator<(const status_code_domain &o) const noexcept;
44
45     constexpr unique_id_type id() const noexcept;
46     constexpr virtual string_ref name() const noexcept = 0;
47     constexpr virtual payload_info_t payload_info() const noexcept = 0;
48
49 protected:
50     constexpr virtual bool _do_failure(const status_code<void> &code) const noexcept = 0;
51     constexpr virtual bool _do_equivalent(const status_code<void> &code1, const status_code<void> &
52         code2) const noexcept = 0;
53     constexpr virtual generic_code _generic_code(const status_code<void> &code) const noexcept = 0;
54     constexpr virtual string_ref _do_message(const status_code<void> &code) const noexcept = 0;
55     [[noreturn]] constexpr virtual void _do_throw_exception(const status_code<void> &code) const = 0;
56     constexpr virtual bool _do_erased_copy(status_code<void> &dst, const status_code<void> &src,
57         payload_info_t dstinfo);
58 };
59 }

```

`string_ref` is an implementation defined type with the public interface of a `string_view` which refers to a string stored somewhere which lasts the lifetime of the program.

`atomic_refcounted_string_ref` is an implementation defined type refining `string_ref` which maintains a thread-safe reference counted shared state containing the string, like a `shared_ptr`. Upon the reference count reaching zero, the resources backing the string are deallocated.

`payload_info_t` describes metadata about the payload which shall be carried by values of `status_code` with this implementation of domain.

This is a minimal example of a valid implementation of `status_code_domain`:

```

1 // The payload for the status_code, which has representation of int
2 enum class arithmetic_errc : int {
3     success = 0,
4     divide_by_zero,
5     integer_divide_overflows,
6     not_integer_division
7 };
8
9 // Forward declare the domain for the payload type, and its status code
10 class _arithmetic_errc_domain;
11 using arithmetic_errc_code = status_code<_arithmetic_errc_domain>;
12
13 // Only final domain types should be marked final!

```

```

14 class _arithmetic_errc_domain final : public status_code_domain {
15     using _base = status_code_domain;
16
17 public:
18     // Typedef value_type to the desired payload type
19     using value_type = arithmetic_errc;
20
21     // Best form is a constructor taking an overridable unique id
22     // (this is more composable later). The ID chosen MUST be VERY
23     // random. Do NOT invent your own ids! Always use a truly random
24     // source such as https://www.random.org/cgi-bin/randbyte?nbytes=8&format=h
25     // to avoid statistical collision.
26     constexpr explicit _arithmetic_errc_domain
27         typename _base::unique_id_type id = 0x290f170194f0c6c7) noexcept
28         : _base(id) {}
29     // Every domain must provide a static constexpr factory function
30     // for itself. As domains are trivially copyable, this is very cheap.
31     static inline constexpr const _arithmetic_errc_domain &get();
32
33     // You must supply the name of the domain
34     constexpr virtual _base::string_ref name() const noexcept override {
35         static string_ref v("arithmetic error domain");
36         return v;
37     }
38     // You must supply metadata about the payload. This implementation
39     // is usually good for 99.9% of payload types and can be copy and pasted.
40     constexpr virtual payload_info_t payload_info() const noexcept override {
41         return {
42             sizeof(value_type),
43             sizeof(status_code_domain *) + sizeof(value_type),
44             (alignof(value_type) > alignof(status_code_domain *))
45             ? alignof(value_type)
46             : alignof(status_code_domain *)});
47     }
48
49 protected:
50     constexpr virtual bool _do_failure(const status_code<void> &code) const noexcept override
51     {
52         assert(code.domain() == *this);
53         const auto &c1 = static_cast<const arithmetic_errc_error &>(code);
54         return c1.value() != arithmetic_errc::success;
55     }
56     constexpr virtual bool _do_equivalent(const status_code<void> &code1,
57                                           const status_code<void> &code2) const noexcept override {
58         // Our status codes are never equivalent to any other in comparisons
59         return false;
60     }
61     constexpr virtual generic_code
62     _generic_code(const status_code<void> &code) const noexcept override {
63         // Our status codes do not map onto generic codes
64         return {};
65     }
66     constexpr virtual _base::string_ref
67     _do_message(const status_code<void> &code) const noexcept override {
68         assert(code.domain() == *this);
69         const auto &c1 = static_cast<const arithmetic_errc_error &>(code);

```

```

70     switch(c1.value())
71     {
72     case arithmetic_errc::success:
73         return _base::string_ref("success");
74     case arithmetic_errc::divide_by_zero:
75         return _base::string_ref("divide by zero");
76     case arithmetic_errc::integer_divide_overflows:
77         return _base::string_ref("integer divide overflows");
78     case arithmetic_errc::not_integer_division:
79         return _base::string_ref("not integer division");
80     }
81     return _base::string_ref("unknown");
82 }
83 [[noreturn]] constexpr virtual void
84 _do_throw_exception(const status_code<void> &code) const override {
85     // Attempting to convert us into a C++ exception throw is not allowed
86     abort();
87 }
88 };
89
90 // Factory implementation
91 constexpr _arithmetic_errc_domain arithmetic_errc_domain;
92 inline constexpr const _arithmetic_errc_domain &arithmetic_errc_domain::get() {
93     return arithmetic_errc_domain;
94 }
95
96 // OPTIONAL: Tell status code via ADL customisation point about the available
97 // implicit conversion such that status code specialisation 'arithmetic_errc_code'
98 // will implicitly construct from all values of type 'arithmetic_errc'
99 inline arithmetic_errc_code make_status_code(arithmetic_errc e) {
100     return arithmetic_errc_code(in_place, e);
101 }

```

Construction and assignment [system.status.code__domain.cons]

```

1     constexpr explicit status_code_domain(unique_id_type id) noexcept;

```

Effects: Constructs an object of class `status_code_domain` whose unique identifier shall be `id`.

Ensures: `id() == id`.

```

1     template<size_t N>
2     constexpr explicit status_code_domain(const char (&uuid)[N]) noexcept;
3     template<size_t N>
4     constexpr explicit status_code_domain(const char* uuid, _uuid_size<N>) noexcept;

```

Effects: Constructs an object of class `status_code_domain` whose unique identifier shall be parsed from a hexadecimal character string of the format {430f1201-94fc-06c7-430f-120194fc06c7} or 430f1201-94fc-06c7-430f-120194fc06c7

Ensures: `id()` is a value derived from parsing the hexadecimal string¹.

Observers [system.status.code__domain.observers]

```
1  constexpr bool operator==(const status_code_domain &o) const noexcept;  
2  constexpr bool operator!=(const status_code_domain &o) const noexcept;  
3  constexpr bool operator<(const status_code_domain &o) const noexcept;
```

Effects: Compares and orders status code domains by their unique identifier.

```
1  constexpr unique_id_type id() const noexcept;
```

Returns: The unique identifier of the status code domain.

```
1  constexpr virtual string_ref name() const noexcept = 0;
```

Returns: The string representation of the status code domain.

```
1  constexpr virtual payload_info_t payload_info() const noexcept = 0;
```

Returns: Metadata about the payload carried by status codes of this domain.

```
1  constexpr virtual bool _do_failure(const status_code<void> &code) const noexcept = 0;
```

Returns: True if the status code's value represents a failure.

```
1  constexpr virtual bool _do_equivalent(const status_code<void> &code1, const status_code<void> &  
    code2) const noexcept = 0;
```

Returns: True if the first status code's value is semantically equivalent to the second status code's value. Semantic equivalence is defined as:

1. If both status codes have equivalent domains to this domain, semantic equivalence is whether the values are equal.
2. Optionally, if the second code's domain is the generic code domain, semantic equivalence is whether this domain considers the first code's value as being equivalent.

¹The reference implementation uses 64 bit integers for the unique identifier, as SG14 decided that was sufficient and 128-bit integers will not become portable until C23 is merged into the C++ IS. The reference implementation XOR combines the first and latter halves of the 128-bit UUID string into a 64 bit unique identifier. If 128-bit integers have become supported in the IS using standard code which works in compile time by the time library implementers implement this proposal, it would be encouraged to use an unsigned 128-bit integer for the unique id type.

- Optionally, this domain may consider values in the second code as equivalent for any other domains of its choosing.

```
1 constexpr virtual generic_code _generic_code(const status_code<void> &code) const noexcept = 0;
```

Returns: The generic code most closely mapping onto the status code's value.

```
1 constexpr virtual string_ref _do_message(const status_code<void> &code) const noexcept = 0;
```

Returns: The string representation of the status code's value.

```
1 [[noreturn]] constexpr virtual void _do_throw_exception(const status_code<void> &code) const = 0;
```

Throws: Throws an exception most closely representing the status code's value.

```
1 constexpr virtual bool _do_erased_copy(status_code<void> &dst, const status_code<void> &src,
    payload_info_t dstinfo);
```

Returns: True if `dst` was successfully configured.

Effects: Using payload metadata from `dstinfo` to configure `dst` correctly, set `dst` to have a value corresponding that from `src`. `dst` is permitted to be empty (and therefore have no domain set). The default implementation retrieves the source payload metadata, and returns false having done nothing if the destination's `total_size` is smaller than the source's `total_size`. If the destination is sufficiently large, `memcpy` is used to copy the representation bits from `src` to `dst`.

Traits [system.status.traits]

```
1 namespace std::system_code {
2     template <class Base, class DomainType> struct mixin : public Base {
3         using Base::Base;
4     };
5 }
```

The `system_code::mixin` template is a customisation point. Users can specialise the template for their status code domain implementation type to cause the contents of the mixin type to be inherited into status codes of that domain. Mixin types may include state, or be stateless.

```
1 namespace std {
2     template <T> struct is_status_code_erasedable {
3         static constexpr bool value = is_trivially_copyable_v<T>;
4     };
5     template <class T> static constexpr bool is_status_code_erasedable_v;
6     template <class T> concept status_code_erasedable = is_status_code_erasedable_v<T>;
7 }
```

This is a trait indicating whether a type is safe for erasure into a trivially copyable type of equal or larger size when stored within a `status_code`. Additional specialisations may be added for user-defined types.

```
1 namespace std {
2     template <class ErasedType>
3     using status_code_erased_tag_type = /* implementation defined */;
4 }
```

This is a convenience type alias making available the implementation defined tag type used to specialise erased editions of status code.

Constraints: `ErasedType` satisfies requirement `TriviallyCopyable`².

```
1 namespace std {
2     template <class T> struct is_status_code;
3     template <class T> static constexpr bool is_status_code_v;
4 }
```

Trait types used to detect `status_code` specialisations.

Quick status codes from enumeration [system.status.quick_status_code_from_enum]

For most users who only wish to wrap an enumeration into a status code domain, there is a convenience facility which generates a custom status code domain from your enumeration.

```
1 namespace std {
2     template <class Enum> class _quick_status_code_from_enum_domain;
3     template <class Enum> using quick_status_code_from_enum_code = status_code<
4         _quick_status_code_from_enum_domain<Enum>>;
5
6     template <class Enum> struct quick_status_code_from_enum_defaults {
7         using code_type = quick_status_code_from_enum_code<Enum>;
8
9         struct mapping {
10             using enumeration_type = Enum;
11
12             const Enum value;
13             const char *message;
14             const std::initializer_list<errc> code_mappings;
15         };
16
17         template <class Base> struct mixin : Base {
18             using Base::Base;
19         };
20
21         // Specialised by user enumeration types
```

²If a trivial relocation or move bitcopying implementation reaches the IS at some future point, trivially relocatable or move bitcopying objects are probably also safe to use here.

```

22     template <class T> struct quick_status_code_from_enum;
23 }

```

An example of use also based around `arithmetic_errc` from the earlier example:

```

1  template <>
2  struct std::quick_status_code_from_enum<arithmetic_errc> :
3      std::quick_status_code_from_enum_defaults<arithmetic_errc> {
4
5      // Text name of the enum
6      static constexpr const auto domain_name = "arithmetic error domain";
7
8      // Unique UUID for the enum. PLEASE use https://www.random.org/cgi-bin/randbyte?nbytes=16&format=h
9      static constexpr const auto domain_uuid = "{be201f65-3962-dd0e-1266-a72e63776a42}";
10
11     // Map of each enum value to its text string, and list of semantically equivalent errc's
12     static const std::initializer_list<mapping> &value_mappings()
13     {
14         // Format is: { enum value, "string representation", { list of errc mappings ... } }
15         static const std::initializer_list<mapping<AnotherCode>> v = {
16             {arithmetic_errc::success, "success", {std::errc::success}},
17             {arithmetic_errc::divide_by_zero, "divide by zero", {std::errc::argument_out_of_domain}},
18             {arithmetic_errc::integer_divide_overflows, "integer divide overflows", {std::errc::
19                 result_out_of_range}},
20             {arithmetic_errc::not_integer_division, "not integer division", {std::errc::invalid_argument}},
21             };
22         return v;
23     }
24
25     // Completely optional definition of mixin for the status code
26     // synthesised from 'arithmetic_errc'. It can be omitted, and the
27     // empty mixin from the defaults base class will be used.
28     template <class Base> struct mixin : Base {
29         using Base::Base;
30         constexpr int custom_method() const { return 42; }
31     };
32 };

```

Class `status_code<void>` [system.status.code.void]

There are two forms of type erased status code, this is the first. This form is always available irrespective of the domain's value type, but cannot be directly copied, moved, nor destructed. Thus one always passes this around by const lvalue reference.

```

1  namespace std {
2      template <> class status_code<void> {
3          template <class T> friend class status_code;
4      public:
5          using domain_type = void;
6          using value_type = void;
7          using string_ref = typename status_code_domain::string_ref;
8      protected:
9          const status_code_domain *_domain{nullptr}; // exposition only

```

```

10
11     constexpr explicit status_code(const status_code_domain *v) noexcept;
12     status_code() = default;
13     status_code(const status_code &) = default;
14     status_code(status_code &&) = default;
15     status_code &operator=(const status_code &) = default;
16     status_code &operator=(status_code &&) = default;
17     ~status_code() = default;
18 public:
19     constexpr const status_code_domain &domain() const noexcept;
20     [[nodiscard]] constexpr bool empty() const noexcept;
21
22     constexpr string_ref message() const noexcept;
23     constexpr bool success() const noexcept;
24     constexpr bool failure() const noexcept;
25
26     template <class T> constexpr bool strictly_equivalent(const status_code<T> &o) const noexcept;
27     template <class T> constexpr bool equivalent(const status_code<T> &o) const noexcept;
28
29     [[noreturn]] constexpr void throw_exception() const;
30 };
31 }

```

Construction and assignment [system.status.code.void.cons]

```

1     constexpr explicit status_code(const status_code_domain *v) noexcept;

```

Effects: Constructs an object of class `status_code<void>` whose domain reference shall be `*v`.

Ensures: `domain() == *v`.

Observers [system.status.code.void.observers]

```

1     constexpr const status_code_domain &domain() const noexcept;

```

Returns: A constant lvalue reference to the domain of this status code.

```

1     [[nodiscard]] constexpr bool empty() const noexcept;

```

Returns: True if this status code was default constructed.

```

1     constexpr string_ref message() const noexcept;

```

Returns: If not empty, `domain()._do_message(*this)`, otherwise the string `"(empty)"`.

```

1     constexpr bool success() const noexcept;

```

Returns: If not empty, ! `domain()._do_failure(*this)`, otherwise false.

```
1 constexpr bool failure() const noexcept;
```

Returns: If not empty, `domain()._do_failure(*this)`, otherwise false.

```
1 template <class T> constexpr bool strictly_equivalent(const status_code<T> &o) const noexcept;
```

Returns: If both empty, true; if neither empty, `domain()._do_equivalent(*this, o)`; otherwise false.

```
1 template <class T> constexpr bool equivalent(const status_code<T> &o) const noexcept;
```

Returns: If both empty, true; if one empty and the other not, false; otherwise the following algorithm is performed:

1. If `domain()._do_equivalent(*this, o)`, return true.
2. If `domain()._do_equivalent(o, *this)`, return true.
3. If `o.domain()._generic_code(o) != errc::unknown` and `domain()._do_equivalent(*this, o.domain()._generic_code(o))`, return true.
4. If `o.domain()._generic_code(o) != errc::unknown` and `domain()._do_equivalent(o.domain()._generic_code(o), *this)`, return true.
5. Otherwise, return false;

```
1 [[noreturn]] constexpr void throw_exception() const;
```

Throws: If not empty, `domain()._do_throw_exception(*this)`, otherwise implementation defined.

Class `status_code<DomainType>` [system.status.code]

This is a typed status code whose payload type is defined by the status code's domain type. The customisation point `system_code::mixin` may inject additional state and/or member functions.

It is required to inherit publicly from `status_code<void>` and `std::system_code::mixin` */*implementation defined base type */*, `DomainType`.

An ADL discovered customisation point `make_status_code(T, Args...)` is looked up by one of the constructors. If it is found, and it generates a status code compatible with this status code, implicit construction from `T, Args...` is made available.

```

1 namespace std {
2     template <class DomainType>
3     requires(
4         (!is_default_constructible_v<typename DomainType::value_type>
5          || is_nothrow_default_constructible_v<typename DomainType::value_type>)
6         && (!is_move_constructible_v<typename DomainType::value_type>
7          || is_nothrow_move_constructible_v<typename DomainType::value_type>)
8         && is_nothrow_destructible_v<typename DomainType::value_type>
9     )
10    class status_code : public /* implementation defined */ {
11        template <class T> friend class status_code;
12    public:
13        using domain_type = DomainType;
14        using value_type = typename domain_type::value_type;
15        using string_ref = typename domain_type::string_ref;
16    protected:
17        value_type _value; // exposition only
18    public:
19        status_code() = default;
20        status_code(const status_code &) = default;
21        status_code(status_code &&) = default;
22        status_code &operator=(const status_code &) = default;
23        status_code &operator=(status_code &&) = default;
24        ~status_code() = default;
25        void swap(status_code &) noexcept(is_nothrow_swappable_v<status_code>);
26
27        template <class T, class... Args,
28            class MakeStatusCodeResult = /* safe ADL lookup of make_status_code() */>
29        requires(!is_same_v<decay_t<T>, status_code> // not copy/move of self
30                && !is_same<decay_t<T>, in_place_t> // not in_place_t
31                && is_status_code_v<MakeStatusCodeResult> // ADL makes a status code
32                && is_constructible_v<status_code, MakeStatusCodeResult> // ADLed status code is
33                    compatible
34        )
35        constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(declval<T>(),
36            declval<Args>()...)));
37
38        template<class Enum, class QuickStatusCodeType
39            = typename quick_status_code_from_enum<Enum>::code_type> // Enumeration has been activated
40        requires(is_constructible_v<status_code, QuickStatusCodeType>) // Its status code is compatible
41        constexpr status_code(Enum &&v) noexcept(is_nothrow_constructible_v<status_code,
42            QuickStatusCodeType::value>);
43
44        template <class... Args>
45        constexpr explicit status_code(in_place_t /*unused */, Args &&... args) noexcept(
46            is_nothrow_constructible_v<value_type, Args &&...>);
47
48        template <class T, class... Args>
49        constexpr explicit status_code(in_place_t /*unused */, initializer_list<T> il, Args &&... args)
50            noexcept(is_nothrow_constructible_v<value_type, initializer_list<T>, Args &&...>);
51
52        constexpr explicit status_code(const value_type &v) noexcept(is_nothrow_copy_constructible_v<
53            value_type>);
54
55        constexpr explicit status_code(value_type &&v) noexcept(is_nothrow_move_constructible_v<value_type>)

```

```

50     ;
51     template <class ErasedType>
52     requires(/* if construction from erased type is safe */)
53     constexpr explicit status_code(const erased_status_code<ErasedType> &v) noexcept(
54         is_nothrow_copy_constructible<value_type>);
55
56     constexpr const domain_type &domain() const noexcept;
57     constexpr string_ref message() const noexcept;
58
59     constexpr value_type &value() & noexcept;
60     constexpr value_type &&value() && noexcept;
61     constexpr const value_type &value() const& noexcept;
62     constexpr const value_type &&value() const&& noexcept;
63
64     constexpr status_code clone() const;
65     constexpr void clear() noexcept;
66 };

```

Construction and assignment [system.status.code.cons]

```

1     status_code() = default;

```

Effects: Constructs an empty `status_code<DomainType>`.

Ensures: `empty()== true`.

```

1     status_code(const status_code &) = default;
2     status_code(status_code &&) = default;
3     status_code &operator=(const status_code &) = default;
4     status_code &operator=(status_code &&) = default;
5     ~status_code() = default;
6     void swap(status_code &) noexcept(is_nothrow_swappable_v<status_code>);

```

Effects: Status code's copy and move constructors and special member functions replicate the availability and noexcept of those of its `value_type`.

Ensures: If not empty, `domain()== DomainType::get()`.

```

1     template <class T, class... Args,
2         class MakeStatusCodeResult = /* safe ADL lookup of make_status_code() */>
3     requires(!is_same_v<decay_t<T>, status_code> // not copy/move of self
4             && !is_same<decay_t<T>, in_place_t> // not in_place_t
5             && is_status_code_v<MakeStatusCodeResult> // ADL makes a status code
6             && is_constructible_v<status_code, MakeStatusCodeResult> // ADLed status code is
7                 compatible
8             )
9     constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(declval<T>(),
10         declval<Args>()...)));

```

Effects: Implicitly constructs an object of class `status_code<DomainType>` from the value yielded by the ADL customisation point `make_status_code(T, Args...)`.

Constraints: `T` cannot be `status_code`; `T` cannot be `in_place_t`; `make_status_code(T, Args...)` must yield a status code from which this status code can be constructed.

Ensures: `domain() == DomainType::get()`.

```
1  template<class Enum, class QuickStatusCodeType
2      = typename quick_status_code_from_enum<Enum>::code_type>    // Enumeration has been activated
3      requires(is_constructible_v<status_code, QuickStatusCodeType>) // Its status code is compatible
4      constexpr status_code(Enum &&v) noexcept(is_nothrow_constructible_v<status_code,
        QuickStatusCodeType>::value);
```

Effects: Implicitly constructs an object of class `status_code<DomainType>` from the enumeration value.

Constraints: `Enum` must have a `quick_status_code_from_enum` specialisation; `quick_status_code_from_enum<Enum>::code_type` must be a type from which this status code can be constructed.

Ensures: `domain() == DomainType::get(); value() == v.`

```
1  template <class... Args>
2  constexpr explicit status_code(in_place_t /*unused */, Args &&... args) noexcept(
    is_nothrow_constructible_v<value_type, Args &&...>);
```

Effects: Explicitly constructs an object of class `status_code<DomainType>` whose `value()` is emplacement constructed from `Args...`

Ensures: `domain() == DomainType::get(); value() == value_type(Args...).`

```
1  template <class T, class... Args>
2  constexpr explicit status_code(in_place_t /*unused */, initializer_list<T> il, Args &&... args)
    noexcept(is_nothrow_constructible_v<value_type, initializer_list<T>, Args &&...>);
```

Effects: Explicitly constructs an object of class `status_code<DomainType>` whose `value()` is emplacement constructed from `il, Args...`

Ensures: `domain() == DomainType::get(); value() == value_type(il, Args...).`

```
1  constexpr explicit status_code(const value_type &v) noexcept(is_nothrow_copy_constructible_v<
    value_type>);
```

Effects: Explicitly constructs an object of class `status_code<DomainType>` whose `value()` is copy constructed from `v`.

Ensures: `domain() == DomainType::get(); value() == v.`

```

1     constexpr explicit status_code(value_type &&v) noexcept(is_nothrow_move_constructible<value_type>)
    ;

```

Effects: Explicitly constructs an object of class `status_code<DomainType>` whose `value()` is move constructed from `v`.

Ensures: `domain() == DomainType::get(); value() == v`.

```

1     template <class ErasedType>
2     requires(/* if construction from erased type is safe */)
3     constexpr explicit status_code(const erased_status_code<ErasedType> &v);

```

Expects: `domain() == v.domain()`.

Effects: Explicitly constructs an object of class `status_code<DomainType>` whose `value()` is bit copied from the erased value type of the erased status code `v`.

Constraints: `value_type` must be `StatusCodeErasable` and `ErasedType` and any mixin must be trivially copyable; `v`'s size must be smaller or equal to `*this`'s size.

Ensures: `domain() == DomainType::get()`.

Observers [system.status.code.observers]

```

1     constexpr const domain_type &domain() const noexcept;

```

Returns: A constant lvalue reference to the domain of this status code.

Remarks: This overrides `status_code<void>::domain()` to return the implementation domain type.

```

1     constexpr string_ref message() const noexcept;

```

Returns: If not empty, `domain()._do_message(*this)`, otherwise the string `"(empty)"`.

Remarks: This overrides `status_code<void>::message()` to return the implementation domain type's `string_ref` rather than the base type.

```

1     constexpr value_type &value() & noexcept;
2     constexpr value_type &&value() && noexcept;
3     constexpr const value_type &value() const& noexcept;
4     constexpr const value_type &&value() const&& noexcept;

```

Returns: A reference to the payload value of this status code.

```
1 constexpr status_code clone() const;
```

Expects: `value_type` is copy constructible.

Returns: A copy of this status code.

Modifiers [system.status.code.modifiers]

```
1 constexpr void clear() noexcept;
```

Effects: If not empty, destroy the payload value and set the status code's value to empty.

Ensures: `empty()== true`.

Typedef `erased_status_code<ErasedType>` [system.status.erased.code]

Constraints: `ErasedType` satisfies requirement `TriviallyCopyable`³.

There are two forms of type erased status code, this is a type alias to the second. This form is move-only, retaining the domain of its source, but erasing the payload value into a trivially copyable storage type if `DomainType::value_type` is `StatusCodeErasable`. The customisation point `system_code::mixin` may inject additional state and/or member functions.

It is required to inherit publicly from `status_code<void>` and `system_code::mixin</*implementation defined base type */, status_code_erased_tag_type<ErasedType>>`.

```
1 namespace std {
2     template <class ErasedType>
3     using erased_status_code = status_code</* implementation defined */>;
4
5     template <class ErasedType>
6     class status_code</* implementation defined */>
7     : public /* implementation defined */ {
8     template <class T> friend class status_code;
9     public:
10         using domain_type = void;
11         using value_type = ErasedType;
12         using string_ref = typename status_code<void>::string_ref;
13     protected:
14         value_type _value; // exposition only
15     public:
16         status_code() = default;
17         status_code(const status_code &) = delete;
18         status_code(status_code &&) = default;
19         status_code &operator=(const status_code &) = delete;
```

³If a trivial relocation or move bitcopying implementation reaches the IS at some future point, trivially relocatable or move bitcopying objects are probably also safe to use here.

```

20     status_code &operator=(status_code &&) = default;
21     constexpr ~status_code();
22
23     template <class DomainType>
24     requires(/* domain value type erasure is safe */
25             && !is_erased_status_code_v<status_code<DomainType>>)
26     constexpr status_code(const status_code<DomainType> &v) noexcept;
27
28     template <class DomainType>
29     requires(/* domain value type erasure is safe */
30             status_code<DomainType> &&v) noexcept;
31
32     template <class T, class... Args,
33             class MakeStatusCodeResult =/* safe ADL lookup of make_status_code() */>
34     requires(!is_same_v<decay_t<T>, status_code> // not copy/move of self
35             && !is_same<decay_t<T>, value_type> // not in_place_t
36             && is_status_code_v<MakeStatusCodeResult> // ADL makes a status code
37             && is_constructible_v<status_code, MakeStatusCodeResult>)) // ADLed status code is
38         compatible
39     )
40     constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(declval<T>()),
41         declval<Args>()...));
42
43     template<class Enum,
44             class QuickStatusCodeType = typename quick_status_code_from_enum<Enum>::code_type>
45             // Enumeration has been activated
46     requires(is_constructible_v<status_code, QuickStatusCodeType> // Its status code is
47             compatible
48     constexpr status_code(Enum &&v) noexcept(is_nothrow_constructible_v<status_code,
49         QuickStatusCodeType>);
50
51     explicit constexpr status_code(const status_code<void> &v);
52
53     constexpr status_code(nothrow_t, const status_code<void> &v) noexcept
54
55     constexpr status_code clone() const;
56     constexpr void clear() noexcept;
57 };
58 }

```

Construction and assignment [system.status.erased.code.cons]

```

1     status_code() = default;

```

Effects: Constructs an empty status code.

Ensures: `empty() == true`.

```

1     status_code(const status_code &) = delete;
2     status_code(status_code &&) = default;
3     status_code &operator=(const status_code &) = delete;
4     status_code &operator=(status_code &&) = default;

```

Effects: Erased status codes are move-only.

Ensures: If the source is not empty, `domain()` becomes the source domain, and the source object becomes empty, otherwise results in an empty status code.

```
1  template <class DomainType>
2  requires(/* domain value type erasure is safe */
3         && !is_erased_status_code_v<status_code<DomainType>>)
4  constexpr status_code(const status_code<DomainType> &v) noexcept;
```

Effects: If the source is not empty, implicitly constructs an object of alias `erased_status_code<ErasedType>` through bit copying of the value of `v`.

Constraints: `value_type` must be `StatusCodeErasable` and `ErasedType` and any mixin must be trivially copyable; `v`'s size must be smaller or equal to `*this`'s size; `status_code<DomainType>` cannot be an erased status code.

Ensures: If source was not empty, `domain()== v.domain()`, otherwise results in an empty status code.

```
1  template <class DomainType>
2  requires(/* domain value type erasure is safe */
3  constexpr status_code(status_code<DomainType> &&v) noexcept;
```

Effects: If the source is not empty, implicitly constructs an object of alias `erased_status_code<ErasedType>` through bit copying of the value of `v`, afterwards the source value is left in a state as if it had a move constructor performed upon it.

Constraints: `value_type` must be `StatusCodeErasable` and `ErasedType` and any mixin must be trivially copyable; `v`'s size must be smaller or equal to `*this`'s size.

Ensures: If not empty, `domain()` becomes the source domain, and the source object becomes empty, otherwise results in an empty status code.

```
1  template <class T, class... Args,
2         class MakeStatusCodeResult = /* safe ADL lookup of make_status_code() */>
3  requires(!is_same_v<decay_t<T>, status_code>           // not copy/move of self
4         && !is_same<decay_t<T>, in_place_t>             // not in_place_t
5         && is_status_code_v<MakeStatusCodeResult>       // ADL makes a status code
6         && is_constructible_v<status_code, MakeStatusCodeResult> // ADLed status code is
           compatible
7  )
8  constexpr status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(declval<T>()),
           declval<Args>()...));
```

Effects: Implicitly constructs an object of the erased status code from the value yielded by the ADL customisation point `make_status_code(T, Args...)`.

Constraints: `T` cannot be `status_code`; `T` cannot be `in_place_t`; `make_status_code(T, Args...)` must yield a status code from which this status code can be constructed.

Ensures: `domain()== make_status_code(T, Args...).domain()`.

```
1  template<class Enum, class QuickStatusCodeType
2      = typename quick_status_code_from_enum<Enum>::code_type>    // Enumeration has been activated
3      requires(is_constructible_v<status_code, QuickStatusCodeType>) // Its status code is compatible
4      constexpr status_code(Enum &&v) noexcept(is_nothrow_constructible_v<status_code,
        QuickStatusCodeType>::value);
```

Effects: Implicitly constructs an object of the erased status code from the enumeration value.

Constraints: `Enum` must have a `quick_status_code_from_enum` specialisation;
`quick_status_code_from_enum<Enum>::code_type` must be a type from which this status code can be constructed.

Ensures: `domain()== make_status_code(T, Args...).domain()`.

```
1  explicit constexpr status_code(const status_code<void> &v);
```

Effects: Explicitly constructs an object of the erased status code from the other kind of erased status code by invoking `v.domain()._do_erased_copy(*this, v, payload_info_t{sizeof(value_type), sizeof(status_code), alignof(status_code)})` to perform the copy.

Ensures: `domain()== v.domain()`.

Throws: An exception if `_do_erased_copy()` returns false.

[Note: On implementations with exceptions globally disabled, one might consider omitting this overload entirely. – end note]

```
1  constexpr status_code(nothrow_t, const status_code<void> &v) noexcept
```

Effects: Constructs an object of the erased status code from the other kind of erased status code by invoking `v.domain()._do_erased_copy(*this, v, payload_info_t{sizeof(value_type), sizeof(status_code), alignof(status_code)})` to perform the copy. If that invocation returns false, the constructed object shall be empty.

Ensures: `domain()== v.domain()` if `_do_erased_copy()` returns true, empty otherwise.

Destructor [system.status.erased.code.cons]

```
1  constexpr ~status_code();
```

Effects: If not empty, invokes `domain()._do_erased_destroy(*this, payload_info_t{sizeof(value_type), sizeof(status_code), alignof(status_code)})`.

Observers [system.status.erased.code.observers]

```
1 constexpr status_code clone() const;
```

Returns: A copy of this erased status code, which if non-empty is obtained by invoking `domain(). _do_erased_copy(ret, *this, domain().payload_info())`.

Throws: An exception if `_do_erased_copy()` returns false.

Modifiers [system.status.erased.code.modifiers]

```
1 constexpr void clear() noexcept;
```

Effects: If not empty, destroy the payload value by invoking `domain(). _do_erased_destroy(*this, payload_info_t{sizeof(value_type), sizeof(status_code), alignof(status_code)})`, and set the status code's value to empty.

Ensures: `empty() == true`.

Status code comparisons [system.status.code.comparisons]

```
1 namespace std {
2     ///! True if the status code's are semantically equal via 'equivalent()'.
3     template<class DomainType1, class DomainType2>
4     constexpr bool operator==(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
5         noexcept;
6
7     ///! True if the status code's are not semantically equal via 'equivalent()'.
8     template<class DomainType1, class DomainType2>
9     constexpr bool operator!=(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
10         noexcept;
11
12     ///! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
13     template<class DomainType1, class T, class MakeStatusCodeResult
14         = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
15         make_status_code(), returns void if not found
16     requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
17         status code
18     constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
19
20     ///! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
21     template<class DomainType1, class T, class MakeStatusCodeResult
22         = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
23         make_status_code(), returns void if not found
24     requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
25         status code
26     constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
27
28     ///! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
29     template<class DomainType1, class T, class MakeStatusCodeResult
```

```

24     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
        make_status_code(), returns void if not found
25 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
        status code
26 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
27
28 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)
    '.
29 template<class DomainType1, class T, class MakeStatusCodeResult
30     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
        make_status_code(), returns void if not found
31 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
        status code
32 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
33
34 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.
35 template <class DomainType1, class T, class QuickStatusCodeType
36     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
37 >
38 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
39
40 //! True if the status code's are semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(a)'.
41 template <class DomainType1, class T, class QuickStatusCodeType
42     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
43 >
44 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
45
46 //! True if the status code's are not semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(b)'.
47 template <class DomainType1, class T, class QuickStatusCodeType
48     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
49 >
50 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
51
52 //! True if the status code's are not semantically equal via 'equivalent()' to '
    quick_status_code_from_enum<T>::code_type(a)'.
53 template <class DomainType1, class T, class QuickStatusCodeType
54     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
55 >
56 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
57 }

```

Exception types [system.status.code.excepts]

For status codes with no natural mapping to an obvious exception type equivalent to their meaning, class `status_error` provides exception types modelling `status_code`. This is a reasonable type to throw from implementations of `status_code_domain::throw_exception()` if nothing more appropriate is to hand.

Class `status_error<void>` [system.status.code.excepts.erased]

```
1 namespace std {
2     template <class DomainType> class status_error;
3
4     template <>
5     class status_error<void> : public std::exception
6     {
7     protected:
8         status_error() = default;
9         status_error(const status_error &) = default;
10        status_error(status_error &&) = default;
11        status_error &operator=(const status_error &) = default;
12        status_error &operator=(status_error &&) = default;
13        ~status_error() override = default;
14
15        virtual const status_code<void> &do_code() const noexcept = 0;
16
17    public:
18        using domain_type = void;
19        using status_code_type = status_code<void>;
20
21    public:
22        const status_code<void> &code() const noexcept { return do_code(); }
23    };
24 }
```

TODO

Class `status_error<DomainType>` [system.status.code.excepts.typed]

```
1 namespace std {
2     template <class DomainType>
3     class status_error : public status_error<void>
4     {
5         status_code<DomainType> _code;
6         typename DomainType::string_ref _msgref;
7
8         virtual const status_code<void> &_do_code() const noexcept override final { return _code; }
9
10    public:
11        using domain_type = DomainType;
12        using status_code_type = status_code<DomainType>;
13
14        explicit status_error(status_code<DomainType> code);
15
16        virtual const char *what() const noexcept override;
17
18        const status_code_type &code() const &;
19        status_code_type &code() &;
20        const status_code_type &&code() const &&;
21        status_code_type &&code() &&;
22    };
23 }
```

TODO

Generic error coding implementation [system.status.code.generic.impl]

```
1 namespace std {
2     class _generic_code_domain; // exposition only
3     using generic_error = status_error<_generic_code_domain>;
4     constexpr _generic_code_domain generic_code_domain;
5     constexpr inline generic_code make_status_code(errc c) noexcept;
6 }
```

The implementation of alias `generic_code` declared before `status_code_domain`. `generic_code` is enabled for implicit construction from `errc` enumeration values.

Class `errored_status_code<DomainType>` [system.status.errored.code]

This refines `status_code<DomainType>` to include a precondition that the value always indicates a failure.

[Note: If Contracts are in the IS by the time this is standardised, it is explicitly intended that construction of the `errored_*` types with a non-failure value is a contract violation.
– end note]

```
1 namespace std {
2     template <class DomainType>
3     class errored_status_code : public status_code<DomainType>
4     {
5         using _base::clear; // disabled in this refinement
6         using _base::success; // disabled in this refinement
7
8     public:
9         ///! The type of the errored error code.
10        using typename status_code<DomainType>::value_type;
11        ///! The type of a reference to a message string.
12        using typename status_code<DomainType>::string_ref;
13
14        ///! Default constructor.
15        errored_status_code() = default;
16        ///! Copy constructor.
17        errored_status_code(const errored_status_code &) = default;
18        ///! Move constructor.
19        errored_status_code(errored_status_code &&) = default;
20        ///! Copy assignment.
21        errored_status_code &operator=(const errored_status_code &) = default;
22        ///! Move assignment.
23        errored_status_code &operator=(errored_status_code &&) = default;
24        ~errored_status_code() = default;
25
26        ///! Explicitly construct from any similar status code
27        constexpr explicit errored_status_code(const _base &o)
28            noexcept(std::is_nothrow_copy_constructible<_base>::value)
29            [[expects: o.failure() == true]];
```

```

30  //! Explicitly construct from any similar status code
31  constexpr explicit errored_status_code(_base &&o)
32      noexcept(std::is_nothrow_move_constructible<_base>::value)
33      [[expects: o.failure() == true]];
34
35  //! Implicit construction from any type where an ADL discovered
36  //! 'make_status_code(T, Args ...)' returns a 'status_code'.
37  template <class T, class... Args, class MakeStatusCodeResult
38      = typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
39      make_status_code(), returns void if not found
40  requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move
41      of self
42      && !std::is_same<typename std::decay<T>::type, in_place_t>::value // not
43      in_place_t
44      && is_status_code<MakeStatusCodeResult>::value // ADL makes a
45      status code
46      && std::is_constructible<errored_status_code, MakeStatusCodeResult>::value // ADLed
47      status code is compatible
48  )
49  constexpr errored_status_code(T &&v, Args &&... args)
50      noexcept(noexcept(make_status_code(std::declval<T>(), std::declval<Args>()...)))
51      [[expects: make_status_code(std::forward<T>(v) /* unsafe? */, std::forward<Args>(args)...).
52      failure() == true]];
53
54  //! Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
55  template<class Enum, class QuickStatusCodeType
56      = typename quick_status_code_from_enum<Enum>::code_type> // Enumeration has been
57      activated
58  requires(std::is_constructible<errored_status_code, QuickStatusCodeType>::value) // Its status
59      code is compatible
60  constexpr errored_status_code(Enum &&v)
61      noexcept(std::is_nothrow_constructible<errored_status_code, QuickStatusCodeType>::value)
62      [[expects: errored_status_code(QuickStatusCodeType(static_cast<Enum &&(v))).failure() == true
63      ]];
64
65  //! Explicit in-place construction.
66  template <class... Args>
67  constexpr explicit errored_status_code(in_place_t /*unused */, Args &&... args)
68      noexcept(std::is_nothrow_constructible<value_type, Args &&...>::value)
69      [[expects: _base(std::forward<Args>(args)... /* unsafe? */).failure() == true]];
70
71  //! Explicit in-place construction from initializer list.
72  template <class T, class... Args>
73  constexpr explicit errored_status_code(in_place_t /*unused */, std::initializer_list<T> il, Args
74      &&... args)
75      noexcept(std::is_nothrow_constructible<value_type, std::initializer_list<T>, Args &&...>::value)
76      [[expects: _base(il, std::forward<Args>(args)... /* unsafe? */).failure() == true]];
77
78  //! Explicit copy construction from a 'value_type'.
79  constexpr explicit errored_status_code(const value_type &v)
80      noexcept(std::is_nothrow_copy_constructible<value_type>::value)
81      [[expects: _base(v).failure() == true]];
82
83  //! Explicit move construction from a 'value_type'.
84  constexpr explicit errored_status_code(value_type &&v)
85      noexcept(std::is_nothrow_move_constructible<value_type>::value)

```

```

76     [[expects: _base(std::move(v) /* unsafe? */.failure() == true)]];
77
78     /*! Explicit construction from an erased status code. Available only if
79     'value_type' is trivially destructible and 'sizeof(status_code) <= sizeof(status_code<erased<>>)'.
80     Does not check if domains are equal.
81     */
82     template <class ErasedType>
83     requires(detail::domain_value_type_erasure_is_safe<domain_type, erased<ErasedType>>::value)
84     constexpr explicit errored_status_code(const status_code<erased<ErasedType>> &v)
85         noexcept(std::is_nothrow_copy_constructible<value_type>::value)
86         [[expects: v.failure() == true]];
87
88     /*! Always false (including at compile time), as errored status codes are never successful.
89     constexpr bool success() const noexcept { return false; }
90     /*! Return a const reference to the 'value_type'.
91     constexpr const value_type &value() const &noexcept;
92 };
93 }

```

TODO

Typedef `erased_errored_status_code<ErasedType>` `[system.status.erased.errored.code]`

This refines `erased_status_code<ErasedType>` to include a precondition that the value always indicates a failure.

[Note: If Contracts are in the IS by the time this is standardised, it is explicitly intended that construction of the `errored_*` types with a non-failure value is a contract violation.
– end note]

```

1 namespace std {
2     template <class ErasedType>
3     using erased_errored_status_code = errored_status_code</* implementation defined */>;
4
5     template <class ErasedType>
6     class errored_status_code</* implementation defined */>
7         : public erased_status_code</* implementation defined */>
8     {
9     using _base = status_code<erased<ErasedType>>;
10    using _base::success;
11
12    public:
13    using domain_type = typename _base::domain_type;
14    using value_type = typename _base::value_type;
15    using string_ref = typename _base::string_ref;
16
17    /*! Default construction to empty
18    errored_status_code() = default;
19    /*! Copy constructor
20    errored_status_code(const errored_status_code &) = default;
21    /*! Move constructor
22    errored_status_code(errored_status_code &&) = default;
23    /*! Copy assignment

```

```

24   errored_status_code &operator=(const errored_status_code &) = default;
25   //! Move assignment
26   errored_status_code &operator=(errored_status_code &&) = default;
27   ~errored_status_code() = default;
28
29   //! Explicitly construct from any similarly erased status code
30   constexpr explicit errored_status_code(const _base &o)
31       noexcept(std::is_nothrow_copy_constructible<_base>::value)
32       [[expects: o.failure() == true]];
33   //! Explicitly construct from any similarly erased status code
34   constexpr explicit errored_status_code(_base &&o)
35       noexcept(std::is_nothrow_move_constructible<_base>::value)
36       [[expects: o.failure() == true]];
37
38   //! Implicit copy construction from any other status code if its value type is
39   //! trivially copyable, it would fit into our storage, and it is not an erased
40   //! status code.
41   template <class DomainType>
42   requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value
43           && !detail::is_erased_status_code<status_code<typename std::decay<DomainType>::type>::
44           value)
45   constexpr errored_status_code(const status_code<DomainType> &v) noexcept
46       [[expects: v.failure() == true]];
47
48   //! Implicit copy construction from any other status code if its value type is
49   //! trivially copyable, it would fit into our storage, and it is not an erased
50   //! status code.
51   template <class DomainType>
52   requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value
53           && !detail::is_erased_status_code<status_code<typename std::decay<DomainType>::type>::
54           value)
55   constexpr errored_status_code(const errored_status_code<DomainType> &v) noexcept;
56
57   //! Implicit move construction from any other status code if its value type is trivially copyable
58   //! or move relocating and it would fit into our storage
59   template <class DomainType>
60   requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value)
61   constexpr errored_status_code(status_code<DomainType> &&v) noexcept
62       [[expects: v.failure() == true]];
63
64   //! Implicit move construction from any other status code if its value type is trivially copyable
65   //! or move relocating and it would fit into our storage
66   template <class DomainType>
67   requires(detail::domain_value_type_erasure_is_safe<erased<ErasedType>, DomainType>::value)
68   constexpr errored_status_code(errored_status_code<DomainType> &&v) noexcept;
69
70   //! Implicit construction from any type where an ADL discovered 'make_status_code(T, Args ...)'
71   //! returns a 'status_code'.
72   template <class T, class... Args,
73           class MakeStatusCodeResult =
74           typename detail::safe_get_make_status_code_result<T, Args...>::type> // Safe ADL lookup of
75           make_status_code(), returns void if not found
76   requires(!std::is_same<typename std::decay<T>::type, errored_status_code>::value // not copy/move
77           of self
78           && !std::is_same<typename std::decay<T>::type, value_type>::value // not copy/move
79           of value type

```

```

75         && is_status_code<MakeStatusCodeResult>::value // ADL makes a
            status code
76         && std::is_constructible<errored_status_code, MakeStatusCodeResult>::value)) // ADLed
            status code is compatible
77     )
78     constexpr errored_status_code(T &&v, Args &&... args) noexcept(noexcept(make_status_code(std::
        declval<T>(), std::declval<Args>()...))) [[expects: make_status_code(std::forward<T>(v) /*
        unsafe? */ , std::forward<Args>(args)...).failure() == true]];
79
80     //! Implicit construction from any 'quick_status_code_from_enum<Enum>' enumerated type.
81     template<class Enum,
82             class QuickStatusCodeType = typename quick_status_code_from_enum<Enum>::code_type)
            // Enumeration has been activated
83     requires(std::is_constructible<errored_status_code, QuickStatusCodeType>::value) // Its
        status code is compatible
84     constexpr errored_status_code(Enum &&v) noexcept(std::is_nothrow_constructible<status_code,
        QuickStatusCodeType>::value);
85
86     //! Explicit copy construction from an unknown status code. Note that this will throw
87     //! an exception if its value type is not trivially copyable or would not
88     //! fit into our storage or the source domain's '_do_erased_copy()' refused the copy.
89     //! This function is not present if C++ exceptions are globally disabled.
90     explicit constexpr errored_status_code(const status_code<void> &v);
91
92     // errored_status_code(std::nothrow_t, const status_code<void> &v) is deliberately omitted,
93     // as empty errored_status_code's are not possible due to contract violation. One can
94     // use status_code's nothrow constructor, do a runtime check for emptiness, then implicitly
95     // construct an errored_status_code from that.
96
97     //! Always false (including at compile time), as errored status codes are never successful.
98     constexpr bool success() const noexcept { return false; }
99     //! Return the erased 'value_type' by value.
100    constexpr value_type value() const noexcept;
101 };
102 }

```

TODO

Errored status code comparisons [system.status.errored.code.comparisons]

```

1 namespace std {
2     //! True if the status code's are semantically equal via 'equivalent()'.
3     template <class DomainType1, class DomainType2>
4     constexpr bool operator==(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
        noexcept;
5     template <class DomainType1, class DomainType2>
6     constexpr bool operator==(const status_code<DomainType1> &a, const errored_status_code<DomainType2>
        &b) noexcept;
7     template <class DomainType1, class DomainType2>
8     constexpr bool operator==(const errored_status_code<DomainType1> &a, const status_code<DomainType2>
        &b) noexcept;
9
10    //! True if the status code's are not semantically equal via 'equivalent()'.
11    template <class DomainType1, class DomainType2>

```

```

12 constexpr bool operator!=(const status_code<DomainType1> &a, const status_code<DomainType2> &b)
    noexcept;
13 template <class DomainType1, class DomainType2>
14 constexpr bool operator!=(const status_code<DomainType1> &a, const errored_status_code<DomainType2>
    &b) noexcept;
15 template <class DomainType1, class DomainType2>
16 constexpr bool operator!=(const errored_status_code<DomainType1> &a, const status_code<DomainType2>
    &b) noexcept;
17
18 //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
19 template<class DomainType1, class T, class MakeStatusCodeResult
20     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
21 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
22 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
23 template<class DomainType1, class T, class MakeStatusCodeResult
24     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
25 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
26 constexpr bool operator==(const errored_status_code<DomainType1> &a, const T &b);
27
28 //! True if the status code's are semantically equal via 'equivalent()' to 'make_status_code(T)'.
29 template<class DomainType1, class T, class MakeStatusCodeResult
30     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
31 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
32 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
33 template<class DomainType1, class T, class MakeStatusCodeResult
34     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
35 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
36 constexpr bool operator==(const T &a, const errored_status_code<DomainType1> &b);
37
38 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
39 template<class DomainType1, class T, class MakeStatusCodeResult
40     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
41 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
42 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
43 template<class DomainType1, class T, class MakeStatusCodeResult
44     = typename detail::safe_get_make_status_code_result<const T &>::type> // Safe ADL lookup of
    make_status_code(), returns void if not found
45 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
    status code
46 constexpr bool operator!=(const errored_status_code<DomainType1> &a, const T &b);
47
48 //! True if the status code's are not semantically equal via 'equivalent()' to 'make_status_code(T)'.
49 template<class DomainType1, class T, class MakeStatusCodeResult

```

```

50     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
        make_status_code(), returns void if not found
51 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
        status code
52 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
53 template<class DomainType1, class T, class MakeStatusCodeResult
54     = typename detail::safe_get_make_status_code_result<const T &::type> // Safe ADL lookup of
        make_status_code(), returns void if not found
55 requires(is_status_code<MakeStatusCodeResult>::value) // ADL makes a
        status code
56 constexpr bool operator!=(const T &a, const errored_status_code<DomainType1> &b);
57
58 //! True if the status code's are semantically equal via 'equivalent()' to '
        quick_status_code_from_enum<T>::code_type(b)'.
59 template <class DomainType1, class T, class QuickStatusCodeType
60     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
61 >
62 constexpr bool operator==(const status_code<DomainType1> &a, const T &b);
63 template <class DomainType1, class T, class QuickStatusCodeType
64     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
65 >
66 constexpr bool operator==(const errored_status_code<DomainType1> &a, const T &b);
67
68 //! True if the status code's are semantically equal via 'equivalent()' to '
        quick_status_code_from_enum<T>::code_type(a)'.
69 template <class DomainType1, class T, class QuickStatusCodeType
70     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
71 >
72 constexpr bool operator==(const T &a, const status_code<DomainType1> &b);
73 template <class DomainType1, class T, class QuickStatusCodeType
74     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
75 >
76 constexpr bool operator==(const T &a, const errored_status_code<DomainType1> &b);
77
78 //! True if the status code's are not semantically equal via 'equivalent()' to '
        quick_status_code_from_enum<T>::code_type(b)'.
79 template <class DomainType1, class T, class QuickStatusCodeType
80     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
81 >
82 constexpr bool operator!=(const status_code<DomainType1> &a, const T &b);
83 template <class DomainType1, class T, class QuickStatusCodeType
84     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
85 >
86 constexpr bool operator!=(const errored_status_code<DomainType1> &a, const T &b);
87
88 //! True if the status code's are not semantically equal via 'equivalent()' to '
        quick_status_code_from_enum<T>::code_type(a)'.
89 template <class DomainType1, class T, class QuickStatusCodeType
90     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
91 >
92 constexpr bool operator!=(const T &a, const status_code<DomainType1> &b);
93 template <class DomainType1, class T, class QuickStatusCodeType
94     = typename quick_status_code_from_enum<T>::code_type // Enumeration has been activated
95 >
96 constexpr bool operator!=(const T &a, const errored_status_code<DomainType1> &b);
97 }

```

TODO

Quick declaration of a new status code domain implementation [system.status.code.quick.impl]

```
1 namespace std {
2     template <class Enum>
3     class _quick_status_code_from_enum_domain<Enum>; // exposition only
4     template <class Enum>
5     constexpr _quick_status_code_from_enum_domain<Enum> quick_status_code_from_enum_domain = {};
6
7     namespace mixins {
8         template <class Base, class Enum>
9         struct mixin<Base, _quick_status_code_from_enum_domain<Enum>> : public quick_status_code_from_enum
10             <Enum>::template mixin<Base> {
11             using quick_status_code_from_enum<Enum>::template mixin<Base>::mixin;
12         };
13     } // namespace mixins
14 }
```

The implementation of the coding domain for each enumeration `Enum`. As `Enum` is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(Enum)` is enabled.

error_category compatibility [system.status.code.error_category]

Status codes will implicitly construct from an `error_code`, using a status code domain which wraps the error code's category and delegates decisions to it in a backwards compatible way. The alias to the typed form of status codes representing an `error_code` is `system_error_code`. As `error_code`'s value type is `int` which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(int)` is enabled.

```
1 namespace std {
2     class _std_error_code_domain; // exposition only
3     namespace mixins {
4         template <class Base>
5         struct mixin<Base, _std_error_code_domain> : public Base {
6             using Base::Base;
7             inline mixin(std::error_code ec);
8
9             // Returns the original category of the original error code
10             inline const std::error_category &category() const noexcept;
11         };
12     } // namespace mixins
13     using system_error_code = status_code</* implementation defined */>;
14 }
```

POSIX coding [system.status.code.posix_code]

POSIX codes are whatever superset of POSIX <errno> the operating system implements. For convenience, a mixin `posix_code::current()` is provided which returns a status code with the posix code domain containing the current value of POSIX `errno`. As POSIX `errno`'s type is `int` which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(int)` is enabled.

If the implementation does not implement any form of POSIX support, this section is omitted.

```
1 namespace std {
2     class _posix_code_domain; // exposition only
3     namespace mixins {
4         template <class Base>
5         struct mixin<Base, _posix_code_domain> : public Base {
6             using Base::Base;
7
8             //! Returns a 'posix_code' for the current value of 'errno'.
9             static posix_code current() noexcept;
10        };
11    } // namespace mixins
12
13    using posix_code = status_code<_posix_code_domain>;
14    using posix_error = errored_status_code<_posix_code_domain>;
```

HTTP status coding [system.status.code.http_status_code]

HTTP status codes are defined by the IETF RFC 9110 standard, and can represent success, informational, and failure codes. As HTTP status code's type is `int` which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(int)` is enabled.

If the implementation does not implement any form of network support, this section is omitted.

```
1 namespace std {
2     class _http_status_code_domain; // exposition only
3     namespace mixins {
4         template <class Base>
5         struct mixin<Base, _http_status_code_domain> : public Base {
6             using Base::Base;
7
8             //! True if the HTTP status code is informational
9             inline bool is_http_informational() const noexcept;
10            //! True if the HTTP status code is successful
11            inline bool is_http_success() const noexcept;
12            //! True if the HTTP status code is redirection
13            inline bool is_http_redirection() const noexcept;
14            //! True if the HTTP status code is client error
15            inline bool is_http_client_error() const noexcept;
16            //! True if the HTTP status code is server error
17            inline bool is_http_server_error() const noexcept;
18        };
19    } // namespace mixins
20
```

```

21 using http_status_code = status_code<_http_status_code_domain>;
22 using http_status_error = errored_status_code<_http_status_code_domain>;

```

IP address resolution coding [system.status.code.getaddrinfo__code]

Retrieving information about an internet address is very widely supported across implementations and uses a function returning its own error coding which is nearly identical everywhere. As `getaddrinfo()`'s coding type is `int` which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(int)` is enabled.

If the implementation does not implement any form of network support, this section is omitted.

```

1 namespace std {
2     class _getaddrinfo_code_domain; // exposition only
3     //! A getaddrinfo error code, those returned by 'getaddrinfo()'.
4     using getaddrinfo_code = status_code<_getaddrinfo_code_domain>;
5     //! A specialisation of 'errored_status_code' for the getaddrinfo code domain.
6     using getaddrinfo_error = errored_status_code<_getaddrinfo_code_domain>;
7 }

```

Microsoft Windows coding [system.status.code.microsoft]

Microsoft Windows has multiple system error code domains, the most common three of which are encoded here:

1. Win32 error codes, type is `DWORD` which is `unsigned int`. These can reflect either success or cause of failure.
2. NT kernel status codes, type is `NTSTATUS` which is `long`. These can reflect success values, information values, warning values, and error values.
3. COM result codes, type is `HRESULT` which is `long`. These can reflect values of application defined meaning, and codes can be from multiple unknown applications, however they do have a single universal bit which indicates whether a value means success or failure.

All of these representation types are trivially copyable types, so erasure into `erased_status_code` with an erased type larger or equal to `sizeof(representation_type)` is enabled.

If the implementation does not implement any specific form of an error coding listed here, that specific coding is omitted.

```

1 namespace std {
2     class _win32_code_domain; // exposition only
3     class _nt_code_domain;    // exposition only
4     class _com_code_domain;   // exposition only
5     namespace mixins {
6         template <class Base>
7         struct mixin<Base, _win32_code_domain> : public Base {
8             using Base::Base;
9         };
10    };

```

```

10     ///! Returns a 'win32_code' for the current value of 'GetLastError()'.
11     static inline win32_code current() noexcept;
12 };
13 } // namespace mixins
14
15 ///! (Windows only) A Win32 error code, those returned by 'GetLastError()'.
16 using win32_code = status_code<win32_code_domain>;
17 ///! (Windows only) A specialisation of 'errored_status_code' for the Win32 error code domain.
18 using win32_error = errored_status_code<win32_code_domain>;
19
20 ///! (Windows only) A NT error code, those returned by NT kernel functions.
21 using nt_code = status_code<nt_code_domain>;
22 ///! (Windows only) A specialisation of 'errored_status_code' for the NT error code domain.
23 using nt_error = errored_status_code<nt_code_domain>;
24
25 /*! (Windows only) A COM error code. Note semantic equivalence testing is only
26 implemented for 'FACILITY_WIN32' and 'FACILITY_NT_BIT'. As you can see at
27 [https://blogs.msdn.microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/](https://blogs.msdn.
28 microsoft.com/eldar/2007/04/03/a-lot-of-hresult-codes/),
29 there are an awful lot of COM error codes, and keeping mapping tables for all of
30 them would be impractical (for the Win32 and NT facilities, we actually reuse the
31 mapping tables in 'win32_code' and 'nt_code'). You can, of course, inherit your
32 own COM code domain from this one and override the '_equivalent()' function
33 to add semantic equivalence testing for whichever extra COM codes that your
34 application specifically needs.
35 */
36 using com_code = status_code<com_code_domain>;
37 ///! (Windows only) A specialisation of 'errored_status_code' for the COM error code domain.
38 using com_error = errored_status_code<com_code_domain>;
39 }

```

Erased system code and `error` object [system.status.code.aliases]

It is very common to erase system codes and enumerations into a single erased status code type which appears in public interfaces. These type aliases are guaranteed to be sufficiently large to erase any status code constructed using default arguments where those arguments have defaults defined in header `<system_status>`.

```

1 namespace std {
2     using system_code = erased_status_code</* implementation defined, usually an intptr_t on most
3     platforms */>;
4     using error = erased_errored_status_code<system_code::value_type>;
5 }

```

[*Note:* These are an obvious source of potential future ABI breakage, so the erased type ought to be chosen carefully as it can never be changed again without breaking the ABI of everything which returns system codes. – end note]

Exception ptr compatibility [system.status.code.exception_ptr]

```

1 namespace std {
2     system_code system_code_from_exception(
3         std::exception_ptr &&ep = std::current_exception(),
4         system_code not_matched = generic_code(errc::resource_unavailable_try_again)) noexcept;
5 }

```

Returns: If when rethrown `ep`'s exception type would be caught by a standard exception type with a natural mapping onto a status code defined in header `<system_status>`, and if relevant that exception's value has a natural mapping onto a value within a status code defined in header `<system_status>`, return the closest status code with (if relevant) the closest value to the exception. If unable to match, return `not_matched`.

[Note: For example, `bad_alloc` (which is valueless) and `system_error` (which has a value) with `.code() == errc::not_enough_memory` would both return `make_status_code(errc::not_enough_memory)` which returns a `generic_code` which will then implicitly erase itself into `system_code`. Comparisons of that returned system code to `errc::not_enough_memory` will thereafter be true. – end note]

```

1 namespace std {
2     class _std_exception_ptr_domain; // exposition only
3     namespace mixins {
4         template <class Base>
5         struct mixin<Base, _std_exception_ptr_domain> : public Base {
6             using Base::Base;
7
8             // Returns the original exception_ptr
9             std::exception_ptr ptr() const noexcept;
10        };
11    } // namespace mixins
12    using exception_ptr_code = status_code<_std_exception_ptr_domain>;
13 }

```

Status codes will implicitly construct from an `exception_ptr`, using a status code domain which wraps the exception ptr and delegates decisions to it such as whether it is semantically equivalent to another status code. The alias to the typed form of status codes representing an `exception_ptr` is `exception_ptr_code`. As `exception_ptr_code`'s value type is a pointer which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(void *)` is enabled.

iostream support [system.status.code.iostream]

```

1 namespace std {
2     ostream &operator<<(ostream &s, const status_code_domain::string_ref &v);
3 }

```

Effects: Equivalent to `return s << string_view(v);`.

```

1 namespace std {

```

```

2   template <class DomainType>
3   ostream &operator<<(ostream &s, const status_code<DomainType> &v);
4 }

```

Expects: That `DomainType::value_type` implements `ostream &operator<<(ostream &, const value_type &)`.

Effects: If empty, equivalent to `return s << "(empty)";`, otherwise equivalent to `return s << v.domain().name() << ": " << v.value();`.

```

1 namespace std {
2   template <class DomainType>
3   ostream &operator<<(ostream &s, const status_code<DomainType> &v);
4 }

```

Expects: That `DomainType::value_type` is not a valid expression or does not implement `ostream &operator<<(ostream &, const value_type &)`.

Effects: If empty, equivalent to `return s << "(empty)";`, otherwise equivalent to `return s << v.domain().name() << ": " << v.message();`.

Nested status codes [system.status.code.nested]

Nested status codes are status codes which indirect to a dynamically allocated status code, thus enabling a small copyable owning reference to a larger or non-moveable status code in memory. As nested status code's `value_type` is a pointer to the indirected status code which is a trivially copyable type, erasure into `erased_status_code` with an erased type larger or equal to `sizeof(void*)` is enabled. As `erased_status_code` is move-only, expensive dynamic memory allocation copies are not possible after erasure.

```

1 namespace std {
2   template <class T, class Alloc = allocator<decay_t<T>>>
3   requires(is_status_code_v<T>)
4   erased_status_code<add_pointer_t<decay_t<T>>> make_nested_status_code(T &&v, Alloc alloc = {});
5 }

```

Expects: `T` is a status code.

Returns: An erased status code with value type of `T*` pointing to a dynamically allocated status code of type `T` which is allocated and move constructed from `v` using allocator `Alloc`.

Throws: Anything which the allocator may throw.

Ensures: That the member functions of the object referenced by `domain()` invoke the corresponding function in `value()->domain()` for `*value()` for these functions: `name()`, `payload_info()`, `_do_failure()`, `_do_equivalent()`, `_generic_code()`, `_do_message()`, `_do_throw_exception()`; that the domain's `_do_erased_copy()` and `_do_erased_destroy()` invoke the supplied allocator to

perform a new dynamic memory allocation and copy construction, and a destruction and dynamic memory release respectively.

```
1 namespace std {
2     template <class StatusCode, class U>
3     requires(is_status_code_v<StatusCode>>)
4     StatusCode *get_if(status_code<U> *v) noexcept;
5
6     template <class StatusCode, class U>
7     requires(is_status_code_v<StatusCode>>)
8     const StatusCode *get_if(const status_code<U> *v) noexcept;
9 }
```

Expects: `StatusCode` is a status code.

Returns: If `v` is a pointer to a nested status code which indirects to a status code of type `StatusCode`, return a pointer to that status code. Otherwise return null.

```
1 namespace std {
2     template <class StatusCode, class U>
3     requires(is_status_code_v<StatusCode>>)
4     StatusCode &get(status_code<U> &v);
5
6     template <class StatusCode, class U>
7     requires(is_status_code_v<StatusCode>>)
8     const StatusCode &get(const status_code<U> &v);
9 }
```

Expects: `StatusCode` is a status code.

Returns: If `v` is a reference to a nested status code which indirects to a status code of type `StatusCode`, return a reference to that status code.

Throws: If `v` is not a reference to a nested status code, or the nested status code does not indirect to a status code of type `StatusCode`, throws an exception of `bad_nested_status_code_access`.

```
1 namespace std {
2     template <class U>
3     typename status_code_domain::unique_id_type get_id(const status_code<U> &v) noexcept;
4 }
```

Returns: If `v` is a reference to a nested status code, returns the id of the indirected to domain. Otherwise return a meaningless number.

4 Acknowledgements

This paper would not have happened in a timely fashion without my current client MayStreet London Stock Exchange Group permitting me a few work hours per month to write this revision. My thanks to them for enabling this paper revision.

5 References

- [N2066] Beman Dawes,
TR2 Diagnostics Enhancements
<https://wg21.link/N2066>
- [P0262] Lawrence Crowl, Chris Mysisen,
A Class for Status and Optional Value
<https://wg21.link/P0262>
- [P0709] Herb Sutter,
Zero-overhead deterministic exceptions: Throwing values
<https://wg21.link/P0709>
- [P0824] O'Dwyer, Bay, Holmes, Wong, Douglas,
Summary of SG14 discussion on `<system_error>`
<https://wg21.link/P0824>
- [P0829] Ben Craig,
Freestanding proposal
<https://wg21.link/P0829>
- [P1029] Douglas, Niall
move = bitcopies
<https://wg21.link/P1029>
- [P1031] Douglas, Niall
Low level file i/o library
<https://wg21.link/P1031>
- [P1095] Douglas, Niall
Zero overhead deterministic failure – A unified mechanism for C and C++
<https://wg21.link/P1095>
- [P1144] O'Dwyer, Arthur
Object relocation in terms of move plus destroy
<https://wg21.link/P1144>
- [P1195] Dimov, Peter
Making `<system_error>` constexpr
<https://wg21.link/P1195>
- [P1196] Dimov, Peter
Value-based `std::error_category` comparison
<https://wg21.link/P1196>
- [P1197] Dimov, Peter
A non-allocating overload of `error_category::message()`
<https://wg21.link/P1197>

- [P1198] Dimov, Peter
Adding `error_category::failed()`
<https://wg21.link/P1198>
- [P1631] Douglas, Niall and Steagall, Bob
Object detachment and attachment
<https://wg21.link/P1631>
- [P1883] Douglas, Niall
`file_handle` and `mapped_file_handle`
<https://wg21.link/P1883>
- [P2000] H. Hinnant, B. Stroustrup, D. Vandevoorde, M. Wong,
Direction for ISO C++
<http://wg21.link/P2000>
- [P2170] Salvia, Charles
Feedback on implementing the proposed `std::error` type
<https://wg21.link/P2170>
- [1] *Boost.Outcome*
Douglas, Niall and others
<https://ned14.github.io/outcome/>
- [2] *stl-header-heft github analysis project*
Douglas, Niall
<https://github.com/ned14/stl-header-heft>